

Resolução Completa: Modelagem ER, Views SQL e Segurança (DCL)

Este documento contém a resolução completa para os 5 sistemas solicitados. Cada questão possui seu **Diagrama Entidade-Relacionamento (ER) em notação relacional**, explicação da solução, scripts DDL (Criação de Tabelas), DML (População de Dados) e DQL (Views e Testes).

1. Clínica Médica (Proteção de Dados Sensíveis)

Diagrama ER

[PACIENTES] (1) <--- (N) [CONSULTAS] (N) ---> (1) [MEDICOS]

Mapeamento de Atributos:

- PACIENTES (id_paciente [PK], nome, cpf, data_nascimento, telefone, historico_medico_sensivel)
- MEDICOS (id_medico [PK], nome, crm, especialidade)
- CONSULTAS (id_consulta [PK], id_paciente [FK], id_medico [FK], data_consulta, status)

Script SQL

```
CREATE DATABASE clinica_medica;  
USE clinica_medica;
```

```
CREATE TABLE pacientes (  
    id_paciente INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    cpf VARCHAR(14) NOT NULL UNIQUE,  
    data_nascimento DATE NOT NULL,  
    telefone VARCHAR(20),  
    historico_medico_sensivel TEXT  
);
```

```
CREATE TABLE medicos (  
    id_medico INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    crm VARCHAR(20) NOT NULL UNIQUE,  
    especialidade VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE consultas (  
    id_consulta INT AUTO_INCREMENT PRIMARY KEY,  
    id_paciente INT NOT NULL,  
    id_medico INT NOT NULL,  
    data_consulta DATETIME NOT NULL,  
    status VARCHAR(20) DEFAULT 'Agendada',  
    FOREIGN KEY (id_paciente) REFERENCES pacientes(id_paciente),  
    FOREIGN KEY (id_medico) REFERENCES medicos(id_medico)  
);
```

-- População

```
INSERT INTO pacientes (nome, cpf, data_nascimento, telefone,  
historico_medico_sensivel) VALUES  
( 'Ana Silva', '111.222.333-44', '1990-05-12', '(51) 99999-1111', 'Histórico  
de hipertensão severa e ansiedade.' ),  
( 'Bruno Costa', '222.333.444-55', '1985-08-25', '(51) 98888-2222',  
'Acompanhamento de Síndrome de Burnout.' );
```

```
INSERT INTO medicos (nome, crm, especialidade) VALUES  
( 'Dra. Carla Souza', 'CRM/RS 12345', 'Cardiologia' ),  
( 'Dr. Roberto Alves', 'CRM/RS 67890', 'Clínica Geral' );
```

```
INSERT INTO consultas (id_paciente, id_medico, data_consulta, status)  
VALUES  
(1, 1, '2026-09-10 14:00:00', 'Agendada'),  
(2, 2, '2026-09-10 15:00:00', 'Realizada');
```

-- View não sensível para a Recepção (Oculta CPF e Histórico Médico)

```
CREATE VIEW vw_recepcao_pacientes AS  
SELECT  
    p.id_paciente,  
    p.nome AS nome_paciente,  
    p.telefone,  
    c.data_consulta,  
    c.status AS status_consulta,  
    m.nome AS nome_medico,
```

```

        m.especialidade
FROM pacientes p
INNER JOIN consultas c ON p.id_paciente = c.id_paciente
INNER JOIN medicos m ON c.id_medico = m.id_medico;

-- Teste de Seleção
SELECT * FROM vw_recepcao_pacientes;

```

2. Imobiliária (Receita Mensal e Total de Imóveis Alugados por Proprietário)

Diagrama ER

```

[PROPRIETARIOS] (1) <--- (N) [IMOVEIS] (1) <--- (N) [CONTRATOS]

```

Mapeamento de Atributos:

- PROPRIETARIOS (id_proprietario [PK], nome, email, telefone)
- IMOVEIS (id_imovel [PK], id_proprietario [FK], endereco, tipo)
- CONTRATOS (id_contrato [PK], id_imovel [FK], valor_aluguel_mensal, status)

Script SQL

```

CREATE DATABASE imobiliaria;
USE imobiliaria;

```

```

CREATE TABLE proprietarios (
    id_proprietario INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL
);

```

```

CREATE TABLE imoveis (
    id_imovel INT AUTO_INCREMENT PRIMARY KEY,
    id_proprietario INT NOT NULL,
    endereco VARCHAR(200) NOT NULL,
    tipo VARCHAR(50) NOT NULL,
    FOREIGN KEY (id_proprietario) REFERENCES proprietarios(id_proprietario)
);

```

```
);
```

```
CREATE TABLE contratos (  
    id_contrato INT AUTO_INCREMENT PRIMARY KEY,  
    id_imovel INT NOT NULL,  
    valor_aluguel_mensal DECIMAL(10,2) NOT NULL,  
    status VARCHAR(20) DEFAULT 'Ativo',  
    FOREIGN KEY (id_imovel) REFERENCES imoveis(id_imovel)  
);
```

```
-- População
```

```
INSERT INTO proprietarios (nome, email) VALUES  
( 'Carlos Eduardo', 'carlos@email.com'),  
( 'Mariana Dias', 'mariana@email.com');
```

```
INSERT INTO imoveis (id_proprietario, endereco, tipo) VALUES  
(1, 'Rua das Flores, 120 - Ap 301', 'Apartamento'),  
(1, 'Av. Brasil, 500 - Sala 10', 'Sala Comercial'),  
(2, 'Rua Getúlio Vargas, 45', 'Casa');
```

```
INSERT INTO contratos (id_imovel, valor_aluguel_mensal, status) VALUES  
(1, 2500.00, 'Ativo'),  
(2, 1800.00, 'Ativo'),  
(3, 3200.00, 'Encerrado');
```

```
-- View de Receita Mensal por Proprietário
```

```
CREATE VIEW vw_receita_proprietarios AS  
SELECT  
    pr.id_proprietario,  
    pr.nome AS proprietario,  
    COUNT(c.id_contrato) AS total_imoveis_alugados,  
    COALESCE(SUM(c.valor_aluguel_mensal), 0.00) AS receita_mensal_total  
FROM proprietarios pr  
LEFT JOIN imoveis i ON pr.id_proprietario = i.id_proprietario  
LEFT JOIN contratos c ON i.id_imovel = c.id_imovel AND c.status = 'Ativo'  
GROUP BY pr.id_proprietario, pr.nome;
```

```
-- Teste de Seleção
```

```
SELECT * FROM vw_receita_proprietarios;
```

3. Escola (Média por Aluno e Disciplina sem Exportar Outros Alunos)

Diagrama ER

[ALUNOS] (1) <--- (N) [NOTAS] (N) ---> (1) [DISCIPLINAS]

Mapeamento de Atributos:

- ALUNOS (id_aluno [PK], nome, matricula)
- DISCIPLINAS (id_disciplina [PK], nome_disciplina)
- NOTAS (id_nota [PK], id_aluno [FK], id_disciplina [FK], nota)

Script SQL

```
CREATE DATABASE escola;
USE escola;
```

```
CREATE TABLE alunos (
    id_aluno INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    matricula VARCHAR(20) NOT NULL UNIQUE
);
```

```
CREATE TABLE disciplinas (
    id_disciplina INT AUTO_INCREMENT PRIMARY KEY,
    nome_disciplina VARCHAR(100) NOT NULL
);
```

```
CREATE TABLE notas (
    id_nota INT AUTO_INCREMENT PRIMARY KEY,
    id_aluno INT NOT NULL,
    id_disciplina INT NOT NULL,
    nota DECIMAL(4,2) NOT NULL,
    FOREIGN KEY (id_aluno) REFERENCES alunos(id_aluno),
    FOREIGN KEY (id_disciplina) REFERENCES disciplinas(id_disciplina)
);
```

```
-- População
INSERT INTO alunos (nome, matricula) VALUES
('Daniel Oliveira', '2026001'),
```

```

('Fernanda Lima', '2026002');

INSERT INTO disciplinas (nome_disciplina) VALUES
('Banco de Dados'),
('Programação OO');

INSERT INTO notas (id_aluno, id_disciplina, nota) VALUES
(1, 1, 8.5), (1, 1, 9.5),
(1, 2, 7.0), (1, 2, 8.0),
(2, 1, 6.0), (2, 1, 7.0);

-- View com Média Agrupada
CREATE VIEW vw_media_alunos_disciplina AS
SELECT
    a.id_aluno,
    a.nome AS aluno,
    d.nome_disciplina,
    ROUND(AVG(n.nota), 2) AS media_final
FROM alunos a
INNER JOIN notas n ON a.id_aluno = n.id_aluno
INNER JOIN disciplinas d ON n.id_disciplina = d.id_disciplina
GROUP BY a.id_aluno, a.nome, d.id_disciplina, d.nome_disciplina;

-- Teste Geral
SELECT * FROM vw_media_alunos_disciplina;

-- Teste de Privacidade (Simulação do Aluno Daniel acessando apenas os seus
dados)
SELECT * FROM vw_media_alunos_disciplina WHERE id_aluno = 1;

```

4. Rede de Restaurantes (Nota Média dos Pratos)

Diagrama ER

```
[PEDIDOS] (1) <--- (N) [AVALIACOES] (N) ---> (1) [PRATOS]
```

Mapeamento de Atributos:

- PRATOS (id_prato [PK], nome_prato, preco)
- PEDIDOS (id_pedido [PK], data_pedido, mesa)
- AVALIACOES (id_avaliacao [PK], id_prato [FK], id_pedido [FK], nota,

comentario)

Script SQL

```
CREATE DATABASE restaurante;
```

```
USE restaurante;
```

```
CREATE TABLE pratos (  
    id_prato INT AUTO_INCREMENT PRIMARY KEY,  
    nome_prato VARCHAR(100) NOT NULL,  
    preco DECIMAL(8,2) NOT NULL  
);
```

```
CREATE TABLE pedidos (  
    id_pedido INT AUTO_INCREMENT PRIMARY KEY,  
    data_pedido DATETIME NOT NULL,  
    mesa INT NOT NULL  
);
```

```
CREATE TABLE avaliacoes (  
    id_avaliacao INT AUTO_INCREMENT PRIMARY KEY,  
    id_prato INT NOT NULL,  
    id_pedido INT NOT NULL,  
    nota INT CHECK (nota BETWEEN 1 AND 5),  
    comentario TEXT,  
    FOREIGN KEY (id_prato) REFERENCES pratos(id_prato),  
    FOREIGN KEY (id_pedido) REFERENCES pedidos(id_pedido)  
);
```

-- População

```
INSERT INTO pratos (nome_prato, preco) VALUES  
( 'Risoto de Cogumelos', 48.00),  
( 'Filé Mignon com Fritas', 65.00);
```

```
INSERT INTO pedidos (data_pedido, mesa) VALUES  
( '2026-09-01 12:30:00', 4),  
( '2026-09-01 13:00:00', 2);
```

```
INSERT INTO avaliacoes (id_prato, id_pedido, nota, comentario) VALUES  
(1, 1, 5, 'Excelente!'),  
(1, 2, 4, 'Muito bom.');
```

```

(2, 2, 5, 'Perfeito.');
```

```

-- View de Nota Média
CREATE VIEW vw_nota_media_pratos AS
SELECT
    p.id_prato,
    p.nome_prato,
    p.preco,
    COUNT(a.id_avaliacao) AS quantidade_avaliacoes,
    ROUND(AVG(a.nota), 2) AS nota_media
FROM pratos p
LEFT JOIN avaliacoes a ON p.id_prato = a.id_prato
GROUP BY p.id_prato, p.nome_prato, p.preco;

-- Teste
SELECT * FROM vw_nota_media_pratos;
```

5. Sistema de RH (Segregação de Acesso via View + GRANT)

Diagrama ER

```
[CARGOS] (1) <--- (N) [FUNCIONARIOS]
```

Mapeamento de Atributos:

- CARGOS (id_cargo [PK], nome_cargo)
- FUNCIONARIOS (id_funcionario [PK], nome, id_cargo [FK], salario)

Explicação da Solução e Script SQL

Para garantir que o setor de RH veja apenas Nome e Cargo enquanto o setor Financeiro tem acesso também ao Salário, aplicamos a técnica de **Segregação de Colunas por Visões** combinada com o comando DCL GRANT:

```

CREATE DATABASE sistema_rh;
USE sistema_rh;
```

```

CREATE TABLE cargos (
```

```

    id_cargo INT AUTO_INCREMENT PRIMARY KEY,
    nome_cargo VARCHAR(100) NOT NULL
);

CREATE TABLE funcionarios (
    id_funcionario INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    id_cargo INT NOT NULL,
    salario DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (id_cargo) REFERENCES cargos(id_cargo)
);

-- População
INSERT INTO cargos (nome_cargo) VALUES ('Desenvolvedor'), ('Gerente de
Projetos');
INSERT INTO funcionarios (nome, id_cargo, salario) VALUES
('Gabriel Santos', 1, 7500.00),
('Helena Martins', 2, 12000.00);

-- View 1: Sem Salário (Público / RH Geral)
CREATE VIEW vw_funcionarios_publico AS
SELECT
    f.id_funcionario,
    f.nome AS funcionario,
    c.nome_cargo AS cargo
FROM funcionarios f
INNER JOIN cargos c ON f.id_cargo = c.id_cargo;

-- View 2: Com Salário (Setor Financeiro)
CREATE VIEW vw_funcionarios_financeiro AS
SELECT
    f.id_funcionario,
    f.nome AS funcionario,
    c.nome_cargo AS cargo,
    f.salario
FROM funcionarios f
INNER JOIN cargos c ON f.id_cargo = c.id_cargo;

-- Comandos DCL de Permissão (GRANT):
-- 1. Usuários
CREATE USER 'usuario_rh'@'%' IDENTIFIED BY 'SenhaRH123!';
CREATE USER 'usuario_financeiro'@'%' IDENTIFIED BY 'SenhaFin123!';

```

```
-- 2. Concessão de Privilégios Restritos
GRANT SELECT ON sistema_rh.vw_funcionarios_publico TO 'usuario_rh'@'%';
GRANT SELECT ON sistema_rh.vw_funcionarios_financeiro TO
'usuario_financeiro'@'%';

FLUSH PRIVILEGES;

-- Testes
SELECT * FROM vw_funcionarios_publico;
SELECT * FROM vw_funcionarios_financeiro;
```